Nemotron 3.5 Bolt 74B-A9B 24GB: 분석 리포트

게시용 버전 · 원문 https://arxiv.org/abs/2610.33039 · arXiv:2610.33039v1 · NVIDIA · 가중치·런타임 공개(OpenMDW-1.1), 학습 코드 미공개 · 분석일 2026-10-30 · 논문 그림 © NVIDIA · arXiv 라이선스는 CC BY 4.0이지만 PDF에 "© 2026 NVIDIA. All rights reserved."가 함께 있어 게시본에서는 논문 그림을 뺌

핵심 요약

1. **무엇을 하는가:** Nemotron 3.5 Super VL(120B-A12B)을 Star Elastic으로 74B-A9B로 줄이고(층별 expert 수·MoE 폭을 예산 조건부 라우터로 선택), SFT·RL로 회복한 뒤, expert를 2.0625비트 IQ2_XXS로 QAD(Jaccard 손실)해서 텍스트 전용 배포 메모리를 23.83 GB로 맞춘다. LiLiCorr 병렬 초안으로 RTX Spark에서 84 TPS.
2. **파라미터·메모리는 공개 파일과 정확히 맞는다:** config로 센 74.043B / 9.06B가 GGUF(74,042,720,848)와 4,176개 차이이고, Table 2의 구성 요소별 크기가 GGUF 헤더와 모두 0.01 GB 안에서 일치한다.
3. **"24GB"는 텍스트 전용 예산이다:** 비전 1.68 GB를 넣으면 25.52 GB이고, 모델 카드의 공식 레시피는 28GB·32GB(32 GB+ GPU 또는 64 GB Spark)다.
4. **속도는 Mamba가 좌우한다:** 토큰당 읽는 4.82 GB 중 51%가 NVFP4 Mamba 입출력이고, 2비트 routed expert는 9%뿐이다. 2비트 expert는 주로 용량을 줄인다.
5. **경쟁 비교는 대체로 맞지만 선택적이다:** Qwen3.6·Gemma4 대비 각각 11승이지만, GDPval이 더 높은 Qwen3.8 27B는 정확도 표에 없고, "PinchBench #1"은 Table 7과 맞지 않으며, Qwen3.6 MCP-Atlas는 공식값 대신 재현값을 쓴다. 속도는 경쟁 모델보다 13-16% 느리다.
6. **구현 가능성:** NVIDIA GPU 32 GB+·Spark에서는 공식 llama.cpp 레시피로 바로 돈다. **이 맥(24 GiB, Metal)에서 공식 24GB 모델은 사실상 불가능하다.** 메모리가 모자라고 llama.cpp Metal이 NVFP4 행렬곱을 지원하지 않는다(Ollama도 74B-A9B의 Apple Silicon 미지원을 명시). Unsloth IQ1_S(18.5 GB)는 GPU 한도를 올리면 조건부로 가능하고, 같이 나온 E4B(6.2 GB, Gemma-4 계열)는 바로 된다. 학습 파이프라인은 코드가 없어 재현할 수 없다.

01. 논문 분석: Nemotron 3.5 Bolt 74B-A9B 24GB (arXiv:2610.33039)

- 제목: *Nemotron 3.5 Bolt 74B-A9B 24GB: A Hybrid Vision-Language Model for Local AI*
- 저자: NVIDIA (기여자 100여 명, Contributors 절)
- 공개: arXiv v1, 2026-10-15 (cs.CV, 19쪽, 부록 없음) · 연구 페이지 https://research.nvidia.com/labs/nemotron/Nemotron-3.5-Bolt-24GB/ · 가중치 Hugging Face `nvidia/NVIDIA-Nemotron-3.5-Bolt-74B-A9B-*` (OpenMDW-1.1)
- 라이선스: arXiv abs 페이지는 **CC BY 4.0**입니다. 그런데 PDF 첫 쪽 아래에 **"© 2026 NVIDIA. All rights reserved."**가 함께 찍혀 있습니다. 권리 표기가 충돌하므로 논문 그림은 개인용 리포트·PPT에만 쓰고 게시본에서는 뺐습니다 (`thinkworld/` 와 같은 판단).
- 분석 범위: 본문 19쪽, 연구 페이지, Hugging Face 모델 카드와 `config.json` · `hf_quant_config.json` (받은 날짜 2026-10-30), 공개 GGUF 세 개의 **헤더만**(HTTP Range로 텐서 목록과 메타데이터, 가중치는 받지 않음), llama.cpp 소스 일부(`889f2ec`). 학습 코드는 공개되지 않았습니다.

---

1. Abstract (한 줄 요약과 주장)

한 줄 요약: Nemotron 3.5 Super VL(120B 전체, 12B 활성)을 Star Elastic으로 74B-A9B로 줄이고, 후학습으로 회복한 뒤, 전문가(expert) 가중치를 2.0625비트(IQ2_XXS)로 양자화 인지 증류(QAD)해서 **텍스트 전용 배포 메모리를 24 GB 안으로 맞춘** 로컬용 하이브리드 Mamba-Transformer MoE 비전-언어 모델입니다. 병렬 초안 모델 LiLiCorr를 함께 내서 RTX Spark에서 84 TPS를 냅니다.

논문이 내세우는 것은 네 가지입니다(Sec. 1).

1. 메모리 예산을 파라미터 예산으로 바꿔 층별 expert 수와 MoE 폭을 고르는 압축(Star Elastic, Sec. 2).
2. "작은 모델을 높은 비트로"보다 "큰 모델을 낮은 비트로"가 같은 크기에서 더 정확하다는 판단과, 이를 위한 2비트 QAD(Sec. 5).
3. 같은 메모리대(Qwen3.6 35B-A3B, Gemma 4 26B-A4B)보다 에이전트·웹·터미널 벤치마크에서 높은 정확도(Table 7).
4. llama.cpp와 vLLM 양쪽 런타임 지원과 LiLiCorr 초안 모델(Sec. 6-7).

---

2. Method

2.1 전체 흐름 (Fig. 2, Fig. 5, Fig. 7)

```
Nemotron 3.5 Super VL 120B-A12B (40 Mamba / 40 MoE / 8 attention, 512 experts × 폭 2688, top-22)
  │ Star Elastic: 예산 조건부 라우터가 층별 expert 수·MoE 폭을 고름 + 부모 KD (8B 토큰에서 수렴)
  ▼
압축 체크포인트 74B-A9B (층별 E ∈ {240…496}, I ∈ {1536…2560}, top-9) → KD 회복 64B(8K) + 48B(128K) = 총 120B 토큰
  │ 비전: 부모의 비전 타워를 다시 붙이고 projector만 학습(6.6M 이미지) → 비전 중심 SFT → SLERP 80:20
  │ RL: MCP-Atlas RL → RLHF → SWE-RL → (RLHF / 연장 SWE-RL) → SLERP 75:25  = 공개 BF16
  ▼
QAD 1단계: 이전 SWE-RL BF16을 교사로 26B 토큰 (Jaccard 손실)
QAD 2단계: 최종 BF16을 교사로 10M 토큰 (토큰 효율)          = 공개 IQ2 "24GB"
  + LiLiCorr 초안 모델(1.13B, 8칸 블록) → llama.cpp(GGUF) / vLLM(ModelOpt) 배포
```

2.2 압축: Star Elastic (Sec. 2, Eq. 1-3, Table 1)

- **정렬과 접두 마스크:** 보정 데이터 8M 토큰으로 expert(REAP 점수)와 MoE 중간 채널(Star Elastic 점수)을 중요도 순으로 정렬합니다. 후보 크기 c_j마다 "앞에서 c_j개만 남기는" 마스크 m_j[i] = 1{i ≤ c_j}를 둡니다(Eq. 1). 작은 마스크가 큰 마스크의 부분집합이 됩니다.
- **미분 가능한 선택:** 라우터(축마다 MLP 1 → 256 → 40·C_a)가 낸 로짓에 Gumbel-softmax를 씌워 마스크를 가중 평균합니다(soft masking, Fig. 1). 덕분에 KD 손실이 아키텍처 선택까지 미분으로 전달됩니다.
- **예산 손실:** 라우터 가중 파라미터 추정 P(ψ; b)가 목표 b에서 ε = 5% 넘게 벗어날 때만 상대 오차 δ로 벌칙을 줍니다(Eq. 2). 전체 목적은 L_KD + λ·L_budget이고 λ = 1입니다(Eq. 3). 예산 손실은 라우터만, KD는 라우터와 학생을 모두 학습시킵니다.
- **얻은 구조 (Table 1):** Mamba·MoE·attention 층 수, hidden 4096, latent MoE 폭 1024, shared expert 폭 5376은 부모와 같습니다. 층별 routed expert 수 ∈ {240, 304, 368, 432, 496}, MoE 폭 ∈ {1536, 2048, 2304, 2560}이고, top-K는 22 → 9로 고정했습니다.
- **목표:** 배치 1, 128K 문맥에서 24 GB 미만입니다(Sec. 2.3). 학습은 GB200 128장 × 약 64시간입니다.

2.3 비전 적응과 회복 학습 (Sec. 3-4, Fig. 5-6)

- 압축은 **텍스트 타워에만** 적용했고, KD도 텍스트 벤치마크 회복만 겨냥했습니다. 그 결과 압축 직후 비전 점수가 무너집니다(Fig. 6: WebArena 57.4 → 1.35, ScreenSpot-Pro 41.6 → 0, RefCOCO 80.45 → 3.09).
- 비전 projector만 학습(6.6M 이미지, 영상·텍스트 없음)하면 대부분 돌아오고, 비전 중심 SFT(54.1B 유효 토큰, 비전 토큰 47.5%)로 더 회복합니다. 그 뒤 SLERP로 병합합니다.
- RL은 MCP E2E RL(512 배치, MCP-Atlas 48 → 56), RLHF(Nemotron 3 Ultra GenRM), SWE-RL(1,024 궤적/스텝, 최대 196,608 토큰)입니다.

2.4 2비트 양자화와 QAD (Sec. 5, Table 2, Eq. 4)

- routed·shared expert 전체를 IQ2_XXS(2.0625비트)로, Mamba 입출력 projection을 NVFP4(4.5비트)로, attention·latent MoE·임베딩·LM head를 Q8_0(8.5비트)으로, router와 conv를 F32로 둡니다.
- Table 2의 가중치 합은 22.01 GB이고, 런타임(BF16 SSM 상태 0.08, FP8 KV 0.54, 활성 버퍼 0.25)을 더하면 22.87 GB, 초안 모델 0.96을 더하면 23.83 GB입니다. **비전 인코더·projector 1.68 GB는 이 합계에서 제외**합니다(Sec. 8.2).
- **QAD:** 학생(2비트)과 고정된 교사(BF16) 출력 분포의 가중 Jaccard 거리 L = 1 − Σmin(p_s, p_t) / Σmax(p_s, p_t)를 최소화합니다(Eq. 4). KL, 역 KL, JSD, MSE, cosine보다 낫다고 보고합니다(수치 없음). 학습률은 1e-5(1.2B 토큰 warmup)에서 1e-7까지 cosine입니다.

- **2단계:** 1단계는 이전 SWE-RL 체크포인트를 교사로 26B 토큰, 2단계는 최종 BF16을 교사로 10M 토큰입니다(Fig. 7).

2.5 LiLiCorr 초안 모델 (Sec. 6, Table 3-6)

- DFlash식 병렬 초안 본체(5층, 그룹 동적 conv, 1,024 슬라이딩 창 + attention sink) 위에 **2층 상관 헤드(37M)**를 둡니다.
- 블록 B = 8(초안 7칸), 위치마다 후보 K = 8개를 고르고, 인접 위치 후보끼리의 cosine 호환 점수를 한 번에 계산한 뒤, 왼쪽부터 추가 계산 없이 고릅니다.
- 학습 데이터는 7.55M 샘플(입력 19.14B 토큰)이고, GB200 96장(학습 64, 타깃 서빙 32)을 썼습니다. NVFP4 가중치 전용 PTQ로 DGX Spark 처리량이 11% 올라갑니다.

---

3. Experiments

3.1 압축 품질 (Fig. 4)

9개 지표 평균이 부모 57.58에서 압축 체크포인트 54.74로 2.85점(4.94%) 내려갑니다. 이 비교는 SFT·RL 회복과 양자화 **전** 체크포인트입니다.

3.2 양자화 품질과 경쟁 모델 (Table 7, Fig. 8-9)

	Super VL BF16	Bolt BF16	Bolt 24GB (2비트)	Qwen3.6 35B-A3B BF16	Gemma4 26B-A4B	Lightning 30B-A3B
GDPVal-AA-V2	34.48	31.13	**28.47**	25.75	15.33	18.26
Terminal-Bench 2.1	61.94	53.93	**50.98**	44.38	37.22	23.46
BrowseComp	71.50	70.00	**66.75**	48.74	26.30	36.81
PinchBench	90.25	90.35	**87.82**	88.07	74.70	83.43
AA-Omniscience accuracy	25.98	21.80	**16.90**	19.47	22.17	16.63
IFBench	73.04	68.71	**66.17**	63.71	77.25	72.88

(전체 14행은 `04` C4와 `analysis/table_check.py`)

해석:

- **2비트 손실은 대부분 2-5점입니다.** 14개 중 11개에서 BF16 대비 −2.4 ~ −4.9점입니다. 14개 평균은 부모 56.68 → Bolt BF16 53.17 → 2비트 51.09입니다.
- **두 곳에서는 2비트가 BF16보다 높습니다.** AA-Omniscience non-hallucination(+5.13), WebArena(+0.61)입니다. 2단계 QAD가 최종 BF16(RLHF 포함)을 교사로 썼기 때문일 수 있지만, 논문은 설명하지 않습니다.
- **경쟁 모델 대비 승패:** Qwen3.6·Gemma4·Lightning 각각에 대해 비교 가능한 13-14개 중 11개에서 이깁니다. AA-Omniscience accuracy에서는 Qwen3.6과 Gemma4 모두에게 지고, PinchBench에서는 Qwen3.6 BF16(88.07)보다 0.25점 낮습니다.
- **비교 조건이 섞여 있습니다.** 경쟁 모델의 정확도는 BF16으로 쟀습니다(Fig. 9 캡션). 그런데 "같은 메모리대"라는 묶음은 경쟁 모델이 4비트일 때의 크기 기준입니다(연구 페이지). 경쟁 모델에게 유리한 조건이라 Bolt의 우위 주장은 보수적인 쪽입니다.
- **가장 강한 24 GB급 경쟁자가 빠져 있습니다.** 2쪽 그림의 Qwen3.8 27B Q4는 GDPval 약 44로 Bolt(약 28)보다 훨씬 높지만, 속도가 28 TPS라서 Table 8에만 나오고 Table 7 정확도 비교에는 없습니다.

- **Fig. 8과 Table 7 사이에 점수 변화가 있습니다.** QAD 1단계(26B 토큰) 뒤 BrowseComp 69.25가 최종 66.75로 2.5점 내려가고, Terminal-Bench는 48.88 → 50.98로 오릅니다. 2단계(토큰 효율 교사)의 득실은 본문에 정리되어 있지 않습니다.

3.3 초안 모델 (Table 5-6)

SPEED-Bench 880문항에서 평균 수락 길이(AL)는 BF16 drafter 4.736(temp 0), NVFP4 4.681(–1.2%)입니다. 코딩 5.7, 글쓰기 3.7로 범주 차이가 큽니다. 실제 에이전트 재생(AA AgentPerf)에서는 LiLiCorr 4.18 대 DSpark 3.68(+13.6%)입니다.

3.4 속도 (Table 8)

RTX Spark(Surface Laptop Ultra 128 GB)·llama.cpp에서 Bolt 24GB + LiLiCorr는 83.99 TPS입니다. Qwen3.6 35B-A3B Q4 + MTP(96.63), Gemma 4 26B-A4B Q4 + MTP(100.56)보다 13-16% 느리고, 에이전트 재생 시간도 600.8초로 25-32% 깁니다. Qwen3.8 27B + DFlash(28.26 TPS, 1,043초)보다는 빠릅니다.

---

4. Limitations

4.1 저자가 밝힌 것

- 양자화는 "의도적으로 손실이 큽니다". 목표는 BF16에 가까운 것이 아니라 크기 대비 정확도입니다(Sec. 5.1).
- MCP-Atlas는 0.75 임계값 이진 판정이라 RL의 개선이 공식 점수에 반영되지 않았습니다(Sec. 4.2).
- 비전 인코더는 24 GB 목표에서 제외했습니다(Sec. 8.2).

4.2 분석하면서 추가로 발견한 것

1. **"24GB"는 텍스트 전용 예산입니다.** 비전(1.68 GB, GGUF mmproj 실측 1.73 GB)을 넣으면 25.52 GB로 24 GB를 넘습니다. 모델 카드의 공식 실행 레시피도 "28GB"(65K 문맥, q8_0 KV)와 "32GB"(256K)이고, 권장 하드웨어는 32 GB 이상 GPU 또는 64 GB Spark입니다(`04` C1, D3).
2. **디코드 대역폭의 절반은 Mamba 입출력 projection입니다.** 토큰 하나에 읽는 4.82 GB 중 2.47 GB(51%)가 NVFP4 Mamba 가중치이고, 2비트 routed expert는 0.44 GB뿐입니다. 2비트 expert는 주로 **용량**을 줄이고, **속도**는 Mamba 정밀도가 좌우합니다(`03` §2).
3. **"초안 블록 검증 비용 ≈ 토큰 하나"는 라우팅이 겹칠 때만 성립합니다.** 독립·균등 라우팅이면 8토큰 검증은 1.57배의 바이트를 읽습니다(`03` §3).
4. **비교 표가 선택적입니다.** GDPval이 가장 높은 Qwen3.8 27B가 정확도 표에 없습니다. Qwen3.6의 MCP-Atlas는 공식 62.8 대신 저자 재현 51.2를 씁니다. "PinchBench #1"(블로그·연구 페이지)은 Table 7과 맞지 않습니다(`04` C4).
5. **시드·분산 정보가 없습니다.** Table 7, 8은 단일 측정입니다(Table 5만 5시드). 0.25점 차이 같은 순위는 구분하기 어렵습니다.
6. **맥(Apple Silicon)에서 그대로 돌리기 어렵습니다.** llama.cpp의 Metal 백엔드는 NVFP4 행렬곱을 지원하지 않아(`889f2ec`), 디코드 바이트의 절반인 Mamba projection이 CPU로 넘어갑니다. Ollama도 74B-A9B는 Apple Silicon을 지원하지 않는다고 적고 있습니다. 24 GiB 맥은 메모리도 부족합니다(`04` §4).
7. **E4B는 같은 계열이 아닙니다.** 함께 발표된 "Bolt E4B 4GB"는 Gemma-4-E4B를 후학습한 모델(GGUF 아키텍처 `gemma4`, 8B 전체 / 4B 유효)이고, 공식 GGUF는 6.18 GB입니다(`04` D10).

03. 파라미터·메모리 산술, 디코드 대역폭, 초안 검증 비용

계산 스크립트는 네 개입니다. 모두 CPU에서 몇 초 안에 끝나고, 가중치는 받지 않습니다.

- `analysis/gguf_header_check.py` → `gguf_header_results.json` : 공개 GGUF의 헤더만 HTTP Range로 받아 텐서별 형식·바이트를 집계합니다.
- `analysis/config_check.py` → `config_check_results.json` : `config.json`의 층별 블록 설정으로 파라미터, 상태 메모리, 이 맥의 메모리 여유를 계산합니다.
- `analysis/spec_decode_toy.py` → `spec_decode_results.json` : 디코드와 초안 검증의 대역폭 모델입니다.
- `analysis/table_check.py` → `table_check_results.json` : 표·본문 수치를 재계산합니다.

1. 파라미터와 메모리는 공개 파일과 정확히 맞는다

1.1 전체·활성 파라미터

config(`NemotronHPuzzle`)의 88개 블록(Mamba 40, MoE 40, attention 8)을 그대로 세면 다음과 같습니다. 근거는 Table 1과 `config_check.py` 입니다.

구성 요소	식 (층 하나)	전체	토큰당 활성
routed experts	E_l × 2 × 1024 × I_l (relu², gate 없음)	66.14B	9/E_l만 → 1.71B
Mamba 입출력	4096 × 18560 + 8192 × 4096	4.38B	4.38B
shared expert	2 × 4096 × 5376	1.76B	1.76B
임베딩 / LM head	131072 × 4096 (묶지 않음)	0.54B + 0.54B	0 + 0.54B
latent MoE	2 × 4096 × 1024	0.34B	0.34B
attention	4096×4096×2 + 4096×256×2	0.29B	0.29B
router 등	4096 × E_l	0.06B	0.06B
합		**74.043B**	**9.06B** (LM head 빼면 8.52B)

- GGUF 헤더의 `total` 74,042,720,848과 공식 계산의 차이는 4,176개(정규화 벡터 몇 개)입니다. "74B-A9B"는 정확합니다.
- 같은 식에 부모 설정(E = 512, I = 2688, top-22)을 넣으면 120.67B 전체, 12.23B 활성이 나와 "120B-A12B"와도 맞습니다.
- 압축으로 routed expert 파라미터는 부모의 58.7%만 남았고, 토큰당 활성 routed 파라미터는 35.0%로 줄었습니다. top-K 22 → 9의 효과가 큽니다.

1.2 Table 2를 GGUF 헤더로 대조

공개 GGUF(22,018,326,528바이트)의 텐서 843개를 형식별로 묶으면 Table 2와 구성 요소마다 0.01 GB 안에서 일치합니다.

구성 요소	Table 2	GGUF 실측	형식(ggml type)
routed experts	17.05	17.053	IQ2_XXS(16), 80 텐서
Mamba 입출력	2.47	2.466	**type 40 = NVFP4**, 4.5 bpw
임베딩, LM head	0.57 + 0.57	0.570 + 0.570	Q8_0
shared / latent / attention	0.45 / 0.36 / 0.30	0.454 / 0.357 / 0.303	IQ2_XXS / Q8_0 / Q8_0

구성 요소	Table 2	GGUF 실측	형식(ggml type)
router / conv	0.23 / 0.01	0.227 / 0.008	F32
가중치 합	**22.01**	**22.010**	

비전 mmproj GGUF는 0.842B 파라미터(BF16 + F32), 1.734 GB로, Table 2의 1.68 GB보다 0.05 GB 큽니다.

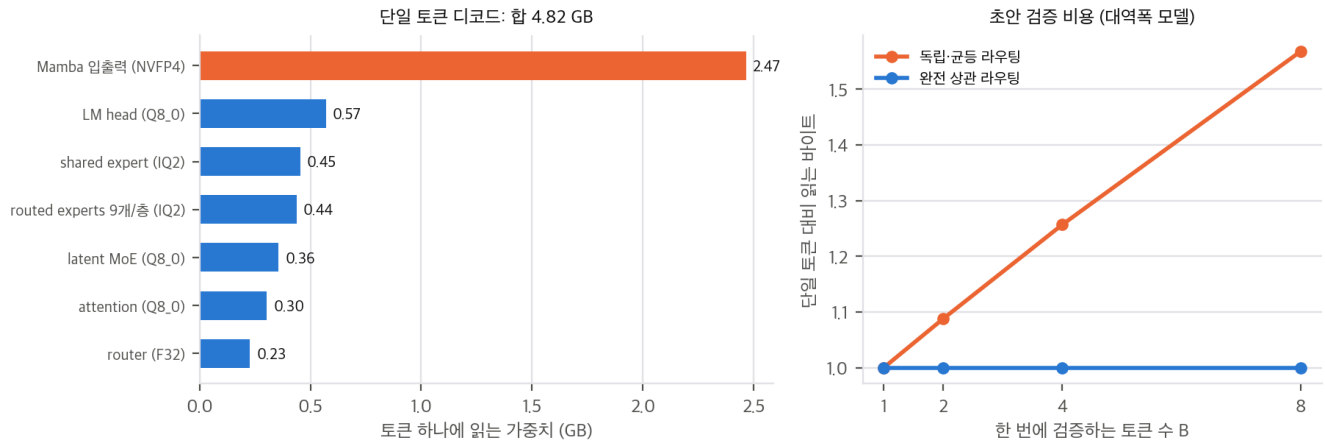
1.3 런타임 상태

- **KV 캐시:** attention 8층 × K·V × KV 헤드 2 × 128차원 = 토큰당 4,096 원소입니다. FP8이면 128K에서 0.537 GB(Table 2의 0.54), llama.cpp 기본인 F16이면 1.07 GB입니다.
- **SSM 상태:** 40층 × (128 헤드 × 64 × 상태 128 + conv 상태 10,240 × 3)입니다. BF16이면 0.086 GB(Table 2의 0.08)입니다. 그런데 config의 `mamba_ssm_cache_dtype`은 `float32`이고, 이때는 0.17 GB입니다.
- Mamba 층 덕분에 문맥이 길어져도 상태가 거의 늘지 않습니다. 256K에서도 KV(F16)가 2.15 GB입니다. **메모리 한계는 문맥이 아니라 가중치가 정합니다.**

2. 디코드 대역폭의 절반은 Mamba 입출력이다

배치 1 디코드는 메모리 대역폭에 묶입니다. 토큰 하나를 만들 때 읽는 가중치를 GGUF 바이트와 층별 expert 수로 계산하면 다음과 같습니다(`spec_decode_toy.py`).

읽는 것	GB	비중
Mamba 입출력 (NVFP4)	2.47	51%
LM head (Q8_0)	0.57	12%
shared expert (IQ2)	0.45	9%
routed experts, 층마다 9개 (IQ2)	**0.44**	**9%**
latent MoE, attention, router	0.89	18%
합	4.82	



GGUF 헤더의 텐서별 바이트와 config의 층별 expert 수로 계산. 실제 라우팅은 두 극단 사이. analysis/spec_decode_toy.py

해석:

- **2비트 expert가 줄이는 것은 주로 용량입니다.** routed expert는 저장 용량의 77%(17.05 / 22.01)를 차지하지만, 토큰당 읽는 양은 9%뿐입니다. 2비트 양자화는 "24 GB 안에 넣기"에 결정적이지만, 속도에는 적게 기여합니다.

- **속도를 좌우하는 것은 Mamba 입출력의 4.5비트입니다.** 이 텐서들까지 2.0625비트로 내리면 토큰당 바이트가 약 28% (2.47 × (1 – 2.0625/4.5) = 1.34 GB) 줄어드는 계산이지만, 논문은 Mamba를 NVFP4에 둔 이유와 그 대가를 설명하지 않습니다.
- 대역폭 273 GB/s(DGX Spark와 M4 Pro의 공칭값으로 알고 있는 값, 확인 필요)를 넣으면, 초안 없이 토큰당 약 57 TPS가 상한입니다. 보고된 84 TPS는 초안 디코딩으로 이 상한을 넘은 값입니다.

3. 초안 블록 검증 비용은 라우팅에 달려 있다

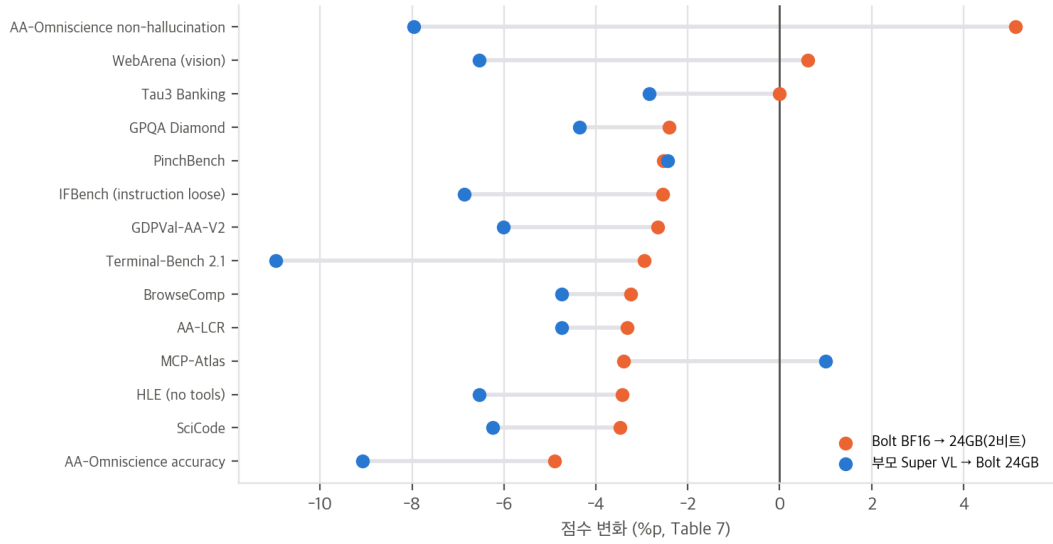
Sec. 6은 "저동시성에서는 가중치 읽기가 병목이라, 초안 블록 검증 비용이 토큰 하나 생성과 거의 같다"고 씁니다. dense 모델이면 맞는 말입니다. MoE에서는 블록의 토큰마다 다른 expert를 고르면 층마다 그 **합집합**을 읽어야 합니다.

토큰 B개가 각자 E_l개 중 9개를 고를 때 기대 고유 expert 수는 E_l(1 – (1 – 9/E_l)^B)입니다(독립·균등 라우팅 가정).

B	독립·균등: 평균 고유 expert	읽는 바이트 (단일 대비)	완전 상관
1	9	1.00	1.00
2	17.8	1.09	1.00
4	34.5	1.26	1.00
8	65.4	**1.57**	1.00

- routed expert가 토큰당 바이트의 9%뿐이라, 최악의 경우에도 8토큰 검증은 1.57배에 그칩니다. 그래서 "거의 같다"는 주장은 이 모델에서는 대체로 맞습니다. **routed expert 비중이 큰 다른 MoE에는 그대로 옮기면 안 됩니다.**
- 대역폭만 보는 속도 향상 상한은 AL × (단일 바이트) / (검증 바이트 + 초안 0.96 GB)입니다. AL 4.18(AA AgentPerf) 기준으로 2.36배(독립)~3.49배(상관)입니다. 실제 라우팅 상관은 공개되지 않아 두 극단만 제시합니다.

4. 2비트의 비용 (Table 7)

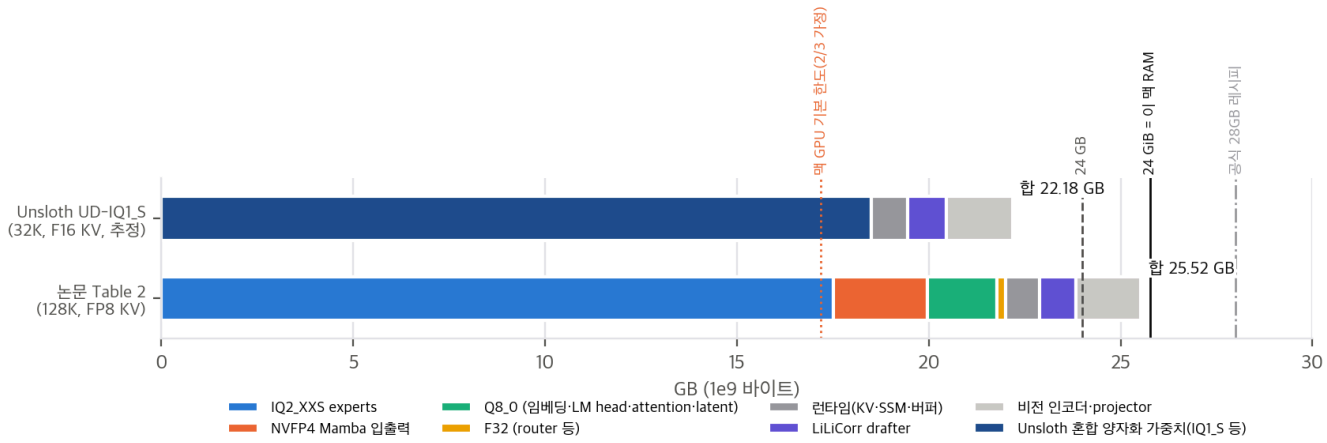


2비트가 BF16보다 높은 곳: AA-Omniscience non-hallucination(+5.13), WebArena(+0.61). 단일 측정, 분산 미보고.

- BF16 → 2비트: 14개 중 11개에서 –2.4 ~ –4.9점, 평균 –2.08점(53.17 → 51.09)입니다.
- 부모 → 2비트: 평균 –5.59점(56.68 → 51.09, 9.9%)이고, Terminal-Bench가 –10.96으로 가장 큽니다. 압축(–3.51)이 양자화(–2.08)보다 큰 손실입니다.

- AA-Omniscience non-hallucination은 BF16 56.14 → 2비트 61.27로 오히려 오릅니다. 같은 지표의 accuracy는 21.80 → 16.90으로 떨어지므로, "모르면 답하지 않는" 쪽으로 성향이 바뀐 것으로 읽힙니다(가설).

5. 메모리 예산 그림



- Table 2의 23.83 GB는 텍스트 전용이고, 비전을 넣으면 25.52 GB입니다. "24 GB"는 1e9 단위 기준이고, 24 GiB(= 25.77 GB) 기준이라면 비전까지 들어가지만 여유는 0.25 GB입니다.
- 모델 카드의 공식 레시피는 28 GB(65K, q8_0 KV)와 32 GB(256K)입니다. 배치 버퍼(batch 2048-4096), 비전, 초안을 모두 포함한 현실적 크기입니다.

04. 공개 모델·런타임 대조, 수치 재계산, 구현 가능성

- **대상:** 학습 코드는 공개되지 않았습니다. 그래서 "코드"는 공개된 산출물로 대신합니다.
  - Hugging Face 설정 파일: BF16 `config.json` (`e09a4ce`), 24GB `config.json` · `hf_quant_config.json` (`cf2effc`), 초안 `config.json` (`449d241`)
  - 공개 GGUF 세 개의 헤더(본 모델 `e0fecb6`, 초안 `182d298`, Unsloth UD-IQ1_S `34841e3`)와 E4B GGUF 헤더 (`1db54d5`)
  - llama.cpp 소스 `889f2ec` (2026-10-30)의 `ggml.h`, `ggml-metal-device.m`, `ggml-cpu.c`
- **범위:** 가중치는 받지 않았습니다. GGUF는 HTTP Range로 앞부분(헤더)만 받았습니다. 모델을 실행하지 않았으므로 속도·정확도는 재현하지 않았습니다.

1. 논문 ↔ 공개 산출물 매핑

논문	공개 산출물	일치 여부
40 Mamba / 40 MoE / 8 attention, hidden 4096 (Table 1)	config `block_configs` 88개, GGUF `block_count` 88	✅
expert 수 ∈ {240, 304, 368, 432, 496}, 폭 ∈ {1536, 2048, 2304, 2560}, top-9	config: E 분포 12/6/9/10/3층, I 분포 1/11/15/13층, top-9	✅ Fig. 3의 층별 순서와도 맞음 (Fig. 3 축의 1,792는 눈금일 뿐 쓰이지 않음)
latent 1024, shared 5376, KV 헤드 2	config `moe_latent_size`, `moe_shared_expert_intermediate_size`, `num_key_value_heads`	✅
74B-A9B, 부모 120B-A12B	식 74.043B / 9.06B, GGUF 74,042,720,848 / 부모 식 120.67B / 12.23B	✅ (`03` §1.1)
Table 2 형식·크기	GGUF 텐서 843개 형식별 합(가중치 22.010 GB)	✅ 구성 요소마다 0.01 GB 안 (`03` §1.2)
IQ2_XXS 2.0625 bpw, Q8_0 8.5, NVFP4 4.5	`hf_quant_config` 27,921항목(ModelOpt 0.37.0 `MIXED_PRECISION`), GGUF bpw	✅
148 GB BF16 → 22.1 GB, 85.1% 감소	74.04B × 2 B = 148.09 GB, GGUF 22.01 GB	✅ (22.01이면 85.13%)
LiLiCorr: B = 8, 후보 K = 8, 헤드 2층·폭 1024·8헤드·FFN 2048, conv (2, 16), 창 1024 + sink	초안 config·GGUF `lilicorr.*` 키, `dflash.*` 키	✅
초안 1.13B(공유 임베딩·LM head 제외), 0.96 GB	초안 GGUF 1,112,828,867 파라미터, HF safetensors 0.9625 GB	⚠️ 파라미터가 1.5% 적음 (D4)
런타임: llama.cpp(GGUF), vLLM(ModelOpt)	llama.cpp PR #28717·#29807·#30506, vLLM PR #57934·#59604 모두 병합	✅

2. 차이와 논문에 적히지 않은 요소

#	내용	영향
D1	**SSM 캐시 정밀도:** Table 2는 BF16(0.08 GB)으로 계산했지만, config의 `mamba_ssm_cache_dtype`은 `float32` 입니다(0.17 GB).	0.09 GB 차이라 예산을 넘기지는 않지만, 0.17 GB 남는 예산(Sec. 8.2) 안에서는 무시할 수 없습니다.
D2	**문맥 길이:** 설계 예산은 128K인데(Sec. 2.3), GGUF·모델 카드는 최대 256K를 광고합니다. 공식 레시피는 65K(28GB) 또는 262K(32GB)입니다.	128K 예산 수치를 256K 사용에 그대로 적용하면 안 됩니다.
D3	**"24GB"와 공식 하드웨어 요구:** 모델 카드의 권장 배포는 "RTX/DGX Spark 64 GB·128 GB 또는 RTX 32 GB+·48 GB+ GPU"이고, 레시피 이름이 28GB·32GB입니다. 24 GB GPU용 레시피는 없습니다.	이름의 24GB는 **텍스트 전용 가중치+상태의 크기**이지, 24 GB 장치에서 돈다는 뜻이 아닙니다(`03` §5).
D4	**초안 모델:** HF config의 `target_layer_ids`는 [1, 18, 35, 51, 68, 85](GGUF는 1 더한 [2, 19, 36, 52, 69, 86]) 여섯 개입니다. 논문은 "다섯 층 + 최종 hidden state"라고 썼는데, 85(86)는 88층 중 마지막이 아닙니다. 파라미터도 1.113B로 논문 1.13B와 1.5% 차이입니다.	사소하지만 재현 시 확인할 부분입니다.
D5	**GGUF의 NVFP4 = ggml type 40:** llama.cpp `889f2ec`의 `ggml_metal_supports_mul_mat_op`는 src0이 NVFP4이면 `false`를 반환하고, `GET_ROWS`도 NVFP4를 제외합니다. CPU 백엔드에는 `ggml_vec_dot_nvfp4_q8_0`(ARM 경로 포함)이 있습니다. 논문이 인용한 SSM 최적화 PR 두 개는 CUDA 전용입니다. 반면 `SSM_SCAN`·`SSM_CONV`는 Metal에서 지원되므로, 막히는 것은 NVFP4뿐입니다. Ollama 라이브러리 페이지(`nemotron-3.5-bolt`, 2026-10-30 확인)도 "Apple Silicon / Metal is not currently supported for 74B-A9B"라고 적고 있습니다(E4B는 지원).	**맥(Metal)에서는 Mamba 입출력 80개 행렬곱이 CPU로 넘어갑니다.** 디코드 바이트의 51%가 여기서 나오므로(`03` §2) 속도 손해가 클 것으로 예상합니다. 실행으로 확인하지는 않았습니다.
D6	`hf_quant_config`의 `kv_cache_quant_algo`는 `None`입니다. Table 2의 FP8 KV는 런타임 설정입니다. llama.cpp 기본값은 F16(128K에서 1.07 GB)이고, 공식 28GB 레시피는 q8_0 KV를 지정합니다.	런타임 설정을 Table 2와 같게 맞추지 않으면 KV가 두 배입니다.
D7	**비전:** config의 비전 타워는 RADIO ViT-H/16(32층, hidden 1280, 최대 13,312 패치, 입력 512)이고, 모델 카드는 C-RADIO v4-H라고 씁니다. mmproj GGUF는 1.734 GB로 Table 2의 1.68 GB보다 0.05 GB 큽니다. 모델 카드는 영상 입력도 지원한다고 쓰지만, 논문의 비전 적응에는 영상 데이터가 없습니다(Sec. 3).	영상 성능은 논문에서 검증되지 않았습니다.
D8	**llama.cpp 레시피의 초안 종류:** 모델 카드 레시피는 `--spec-type draft-dflash`이고 초안 GGUF의 아키텍처도 `dflash`입니다. 그래도 GGUF에 `lilicorr.*` 텐서 26개와 `lilicorr.candidate_topk = 8`이 들어 있고, llama.cpp PR #30506("spec: add lilicorr support")이 병합되어 있습니다.	DFlash 본체 + LiLiCorr 헤드로 읽힙니다. 실제로 헤드가 쓰이는지는 실행해야 확인됩니다.
D9	압축 코드(Star Elastic 라우터), QAD 학습 스크립트, 데이터 혼합 비율, WebArena 내부 하네스가 공개되지 않았습니다.	학습 파이프라인은 재현할 수 없습니다.
D10	**동시 발표된 "Bolt E4B 4GB":** 공식 GGUF의 아키텍처는 `gemma4`이고 파라미터는 7.52B입니다. 모델 카드도 "Gemma-4-E4B-it를 후학습, 8B 전체 / 4B 유효, 64K 문맥"이라고 씁니다. 공식 GGUF(Q4_K_M)는 6.18 GB입니다.	이름은 같은 Bolt지만 하이브리드 Mamba 계열이 아닙니다. "4GB"가 무엇을 뜻하는지 모델 카드만으로는 분명하지 않습니다.

3. 수치 재계산 (`analysis/table_check.py`, `config_check.py`, `spec_decode_toy.py`)

#	주장 (출처)	재계산	판정
C1	24 GB 안에 들어간다 (Abstract, Sec. 8.2)	가중치 22.01 + 런타임 0.87 + 초안 0.96 = 23.84 GB(텍스트 전용). 비전 포함 25.52 GB	텍스트 전용에서만 ✅
C2	74B-A9B, 부모 120B-A12B	74.043B / 9.06B, 부모 120.67B / 12.23B. GGUF와 4,176개 차이	✅
C3	Table 2 구성 요소별 크기	GGUF 헤더와 구성 요소마다 0.01 GB 안	✅
C4a	2비트 손실	BF16 → 24GB 평균 −2.08점(14개), 11개에서 −2.4 ~ −4.9. 두 개는 상승	참고
C4b	같은 메모리대 모델보다 정확 (Abstract, Table 7)	Qwen3.6·Gemma4·Lightning 대비 각각 11승. AA-Omniscience accuracy는 Qwen3.6·Gemma4 모두에게 짐	대체로 ✅
C4c	"PinchBench #1, BrowseComp #2" (블로그·연구 페이지, 같은 메모리대 기준)	Table 7 안에서 PinchBench는 Qwen3.6 BF16 88.07 > Bolt 24GB 87.82	⚠️ Table 7과 맞지 않음. 공개 리더보드 기준으로 보임
C4d	MCP-Atlas 비교	Qwen3.6 공식 62.8 대신 저자 재현 51.2를 써서 Bolt(52.20)가 앞섬. 공식값이면 짐	⚠️ 재현값 의존
C5	84 TPS로 "더 적은 활성 파라미터의 모델과 비슷" (Abstract)	Qwen3.6 Q4 + MTP 대비 0.87배, Gemma4 Q4 + MTP 대비 0.84배. 에이전트 재생 시간은 1.25-1.32배	"비슷"보다는 13-16% 느림
C6	Table 5 Overall = 범주 평균	네 열 모두 ±0.0006 안	✅
C7	NVFP4 초안은 수락 길이를 거의 떨어뜨리지 않음	−1.2%(temp 0), −0.9%(temp 1)	✅
C8	LiLiCorr가 DSpark보다 낫다 (Table 6)	AL +13.6% (4.18 vs 3.68). 본문의 "약 20% 처리량"은 장문맥 별도 실험이고 수치 표가 없음	부분 확인
C9	학습 산술 (Sec. 2.3)	8 + 64 + 48 = 120B 토큰, 배치 2048 × 8K = 128 × 128K = 16.78M 토큰, 128 GPU × 64 h = 8,192 GPU-h	✅
C10	압축 품질 2.85점(4.94%) (Sec. 2.4)	57.58 − 54.74 = 2.84, 2.84/57.58 = 4.93%	✅ (반올림)
C11	QAD 1단계 → 최종 (Fig. 8 vs Table 7)	BrowseComp 69.25 → 66.75, Terminal-Bench 48.88 → 50.98, GDPval 28.99 → 28.47	2단계의 득실이 본문에 없음

4. 구현 가능성 판단

4.1 공개 모델을 그대로 쓰기 (추론)

경로	판단	근거
NVIDIA GPU 32 GB+ 또는 RTX/DGX Spark 64 GB+, llama.cpp	**가능** (공식 지원)	모델 카드 28GB·32GB 레시피, 필요한 llama.cpp PR이 모두 병합됨
NVIDIA GPU 24 GB (예: RTX 4090)	**애매**. 공식 레시피가 없음	텍스트 전용·짧은 문맥·비전 없음이면 가중치 22.0 + 상태 약 0.5 + 버퍼가 24 GB에 걸침
vLLM (SM12x, RTX 5090, Spark)	가능	b12x 커널 PR #59604 병합

4.2 이 맥 (Apple M4 Pro, 통합 메모리 24 GiB = 25.77 GB, CUDA 없음)

`config_check.py`의 계산입니다. **가정:** macOS와 기본 앱에 4 GB를 남기고, llama.cpp compute buffer를 0.5 GB로 둡니다. GPU 기본 한도(`recommendedMaxWorkingSetSize`)는 RAM의 2/3(약 17.2 GB)로 가정했고, 이 맥에서 직접 재지 않았습니다(`iogpu.wired_limit_mb = 0`, 즉 기본값).

파일	가중치	32K 문맥 필요량	GPU 기본 한도 안	RAM − 4 GB 안	판단
공식 24GB IQ2_XXS	22.02 GB	22.96 GB	✗	✗	**사실상 불가.** 메모리가 모자라고, Mamba NVFP4가 Metal에서 돌지 않음(D5)
LM Studio 커뮤니티 GGUF, Ollama `nemotron-3.5-bolt:latest`	22.02 GB / 25 GB(비전 포함)	—	✗	✗	공식 IQ2와 같은 파일 구성이라 같은 판단. Ollama는 Apple Silicon 미지원을 명시
Unsloth UD-IQ1_M	20.31 GB	21.25 GB	✗	✓	조건부. GPU 한도를 올려야 함
Unsloth UD-IQ1_S	18.51 GB	19.45 GB	✗	✓	**조건부 가능.** NVFP4를 쓰지 않아 모든 형식이 Metal 지원 대상. `sudo sysctl iogpu.wired_limit_mb=20480` 정도로 한도를 올리고, 비전·초안 없이, 문맥 32K 이하. 1비트대 품질은 검증되지 않음
커뮤니티 REAP50 13.38 GB	13.38 GB	14.32 GB	✓	✓	들어감. 비공식 추가 가지치기라 품질 미확인
Bolt E4B Q4_K_M	6.18 GB	약 6.7 GB	✓	✓	**바로 가능.** 단, Gemma-4 계열이라 이 논문의 모델이 아님(D10)
MLX 3비트 (rapid-mlx)	31.65 GB	—	✗	✗	불가

결론:

- 이 맥에서 논문의 공식 24GB 모델을 원래 의도대로(GPU에서, 비전과 초안 포함) 돌리는 것은 **불가능합니다.**
- Unsloth IQ1_S를 GPU 한도를 올려 텍스트 전용·짧은 문맥으로 돌리는 것은 **해 볼 수 있습니다.** 다만 메모리 압박이 크고, 품질이 공식판과 다릅니다.
- 실제로 해 보려면 사용자가 확인할 것이 세 가지입니다.
  1. llama.cpp 시작 로그의 `recommendedMaxWorkingSetSize` 값
  2. 한도를 올렸을 때 macOS가 안정적인지
  3. 디스크 여유(18-22 GB)
- 계산상 대역폭 상한(M4 Pro 공칭 273 GB/s, 토큰당 약 4.8 GB 읽음)은 초안 없이 약 57 TPS입니다. 실제로는 더 낮을 것입니다.

4.3 학습 쪽 재현

- 압축(Star Elastic)·QAD 학습 코드가 공개되지 않았고, 규모도 GB200 128장 × 64시간입니다. **재현할 수 없습니다.**
- 공개된 부분으로 해 볼 수 있는 것은 두 가지입니다. ① Model Optimizer로 작은 모델에 PTQ·QAD를 거는 것, ② REAP식 expert 가지치기를 작은 공개 MoE에 적용하는 것. 둘 다 NVIDIA GPU가 필요합니다.

4.4 아이디어만 다시 구현하기

- **메모리 예산 → 파라미터 예산 역산(Sec. 8.2)**과 **텐서별 혼합 정밀도 설계**는 이 저장소의 `config_check.py`처럼 계산 몇십 줄이면 됩니다. 로봇 탑재 VLM의 메모리 계획에 그대로 쓸 수 있습니다.
- Jaccard 손실(Eq. 4)은 한 줄입니다. 효과를 보려면 GPU 학습이 필요합니다.

02. 선행연구와 계보, 공개 저장소

번호 대신 논문 참고문헌의 저자·연도를 씁니다. Star Elastic, Nemotron Elastic, Puzzle 75B-A9B, LiLiCorr는 arXiv abs 제목을 직접 확인했습니다(2026-10-30). 나머지는 본문의 참고문헌을 옮긴 것입니다.

1. 계보

```
[하이브리드 Mamba-Transformer]  Nemotron-H (arXiv:2504.03624) → Nemotron Nano 2 (2508.14444)
        → Nemotron 3 (2512.20856): Nano 30B-A3B / Super 120B-A12B (2604.12374) / Ultra
(2606.15007)
        → Nemotron 3.5 Super VL 120B-A12B (기술 보고서 PDF, arXiv 없음)  ← Bolt의 부모
[압축]  Minitron/Puzzle 계열 → Nemotron Elastic (2511.16664) → Star Elastic (2605.07182)
        Nemotron-Labs-3-Puzzle-75B-A9B (2607.04371): 같은 75B-A9B 급 하이브리드 MoE 압축
[expert 가지치기]  REAP (Lasby et al., ICLR 2026): MoE expert 중요도 점수
[저비트]  ParetoQ (2502.02631), Quest (2502.05003): "큰 모델 저비트 > 작은 모델 고비트"
          QAD (Xin et al., 2026): NVFP4 정확도 회복용 양자화 인지 증류 → 이 논문에서 2비트로 확장
[병렬 초안]  DFlash (Chen et al., ICML 2026) → DSpark (2607.05147) → LiLiCorr (2608.20530) /
DFlash 2 (Inco AI)
          ★ Nemotron 3.5 Bolt 74B-A9B 24GB
```

차별점: 압축(Star Elastic), 2비트 QAD, 병렬 초안을 **하나의 배포 메모리 예산**에 맞춰 묶었다는 점입니다. 개별 기법은 대부분 같은 그룹의 선행 작업이고, 이 보고서의 새로움은 2비트 QAD 확장(Jaccard 손실, 2단계 교사)과 예산 역산(Sec. 8.2)입니다.

2. 구성 요소별 출처

역할	출처	이 논문에서의 쓰임
부모 모델	Nemotron 3.5 Super VL 120B-A12B	압축 대상, KD 교사, 비전 타워
구조 탐색 + 압축	Star Elastic (Taghibakhshi et al., arXiv:2605.07182)	예산 조건부 라우터, Gumbel-softmax 접두 마스크(Eq. 1-3)
expert 정렬	REAP (Lasby et al., ICLR 2026)	routed expert 중요도 순서
2비트 형식	llama.cpp IQ2_XXS (ggml-org 텐서 인코딩)	expert 가중치 2.0625 bpw
4비트 형식	NVFP4 (E4M3 스케일, 16원소 블록)	Mamba 입출력 projection
양자화 인지 증류	QAD (Xin et al., 2026), NVIDIA Model Optimizer	Jaccard 손실로 2비트 확장
병렬 초안	DFlash, DSpark, LiLiCorr (Rusanovsky et al., arXiv:2608.20530)	1.13B 초안, 상관 헤드
비전 인코더	C-RADIO v4-H (config: RADIO ViT-H/16, 32층, hidden 1280)	부모의 비전 타워를 재부착
서빙	llama.cpp(GGUF), vLLM(+vllm-gguf-plugin, b12x 커널)	로컬·고처리량 두 경로
평가	SPEED-Bench, AA-AgentPerf Local, MCP-Atlas, WebArena 등	Table 5-8

3. 이 저장소의 다른 논문과의 관계

솔직하게 말하면 직접 연결은 약합니다. 이 논문은 로봇 학습이 아니라 로컬 LLM/VLM 배포 논문입니다. 연결되는 지점은 다음 정도입니다.

논문	관계
ThinkWorld (`thinkworld/`), **AW-0** (`agentic_wam/`)	둘 다 VLM 백본(Qwen3-VL 계열 등) 위에 행동 헤드를 얹은 VLA입니다. Bolt 같은 압축·저비트 VLM은 **로봇 탑재 추론의 백본 후보**가 될 수 있지만, 이 논문은 로봇 과제나 연속 행동 출력을 전혀 평가하지 않았습니다.
PaceVLA (`pacevla/`)	PaceVLA는 행동 쪽 실행 빈도를 줄여 처리량을 올렸고, Bolt는 언어 쪽 디코드 속도를 초안 모델로 올립니다. 둘 다 "실패를 포함한 처리량"과 "토큰당 지연"을 따로 봐야 한다는 점에서 평가 설계 교훈이 겹칩니다.
Hydra-0 (`hydra0/`)	대규모 생성 모델을 실제 배포 예산에 맞추는 문제의식만 공유합니다.
DexPolicy, FlashDexRetarget 등 RL 논문	공통점은 거의 없습니다. 다만 Bolt의 SWE-RL·MCP RL은 GRPO 계열이라, `dexpolicy/`에서 본 그룹 advantage 설계 문제와 같은 질문을 던질 수 있습니다(이 논문은 advantage 식을 공개하지 않음).

4. 공개 저장소와 모델 (2026-10-30 확인)

Hugging Face (NVIDIA 공식)

저장소	무엇인가	라이선스	다운로드	마지막 수정	확인 revision
nvidia/NVIDIA-Nemotron-3.5-Bolt-74B-A9B-24GB-GGUF	IQ2 GGUF 22.02 GB + mmproj 1.73 GB	OpenMDW-1.1	178,518	2026-10-15	`e0fecb6`
nvidia/NVIDIA-Nemotron-3.5-Bolt-74B-A9B-24GB	같은 IQ2를 HF 형식(ModelOpt `MIXED_PRECISION`, 90 shard, 24.12 GB)	OpenMDW-1.1	12,511	2026-10-20	`cf2effc`
nvidia/NVIDIA-Nemotron-3.5-Bolt-74B-A9B-BF16	BF16 (149.8 GB, 비전 포함)	OpenMDW-1.1	6,520	2026-10-23	`e09a4ce`
nvidia/NVIDIA-Nemotron-3.5-Bolt-74B-A9B-24GB-LiLiCorr	초안 모델 HF (NVFP4/BF16, 0.96 GB)	OpenMDW-1.1	4,829	2026-10-19	`449d241`
nvidia/NVIDIA-Nemotron-3.5-Bolt-74B-A9B-24GB-LiLiCorr-GGUF	초안 모델 GGUF (0.998 GB, 아키텍처 `dflash`)	OpenMDW-1.1	21,949	2026-10-15	`182d298`
nvidia/NVIDIA-Nemotron-3.5-Bolt-E4B-4GB	동시 발표된 소형 모델. Gemma-4-E4B 후학습, GGUF Q4_K_M 6.18 GB	OpenMDW-1.1	64,263	2026-10-23	`1db54d5`

커뮤니티 변환(이 맥 판단에 쓴 것)

저장소	무엇인가	크기
unsloth/NVIDIA-Nemotron-3.5-Bolt-74B-A9B-GGUF (`34841e3`)	UD-IQ1_S·IQ1_M·IQ2_XXS…Q8_0, NVFP4를 쓰지 않는 혼합 양자화	18.51 / 20.31 / 22.02 GB …
rapid-mlx/NVIDIA-Nemotron-3.5-Bolt-74B-A9B-MLX-3bit (`7816420`)	MLX 3비트	31.65 GB
anik-jha/Nemotron-3.5-Bolt-74B-A9B-16GB-GGUF (`e24fd75`)	REAP로 expert를 더 잘라낸 비공식 변형	13.38 GB(REAP50), 9.06 GB(REAP25)
lmstudio-community/NVIDIA-Nemotron-3.5-Bolt-74B-A9B-GGUF (`2116154`), …-LiLiCorr-GGUF	LM Studio용 재배포. IQ2_XXS 파일 크기가 NVIDIA GGUF와 64바이트 차이 → 같은 NVFP4 구성으로 보임	22.02 GB + mmproj 1.73 GB

저장소	무엇인가	크기
Ollama 라이브러리 `nemotron-3.5-bolt` (`:latest`, `:e4b`)	74B-A9B는 25 GB(256K, 텍스트·이미지), E4B는 7.2-7.9 GB. 페이지에 "Apple Silicon / Metal is not currently supported for 74B-A9B" 명시	25 GB / 7.2-7.9 GB

GitHub

저장소	역할	라이선스	★	마지막 push	확인 커밋
ggml-org/llama.cpp	GGUF 추론. 논문이 인용한 PR #28717·#29807(CUDA SSM), #30506(LiLiCorr) 모두 병합	MIT	132,831	2026-10-30	`889f2ec` (소스 일부 읽음)
vllm-project/vllm	고처리량 서빙. PR #57934(LiLiCorr), #59604(IQ2_XXS·Q8_0 b12x) 병합	Apache-2.0	95,910	2026-10-30	`8493b34` (메타데이터만)
vllm-project/vllm-gguf-plugin	vLLM GGUF 실행	Apache-2.0	55	2026-10-24	메타데이터만
local-inference-lab/b12x	SM12x(DGX/RTX Spark, RTX 5090)용 IQ2·Q8_0 커널	Apache-2.0	268	2026-10-06	메타데이터만
NVIDIA/Model-Optimizer	QAD·PTQ, 초안 학습	Apache-2.0	5,471	2026-10-30	`cb9ab4d` (메타데이터만)
NVIDIA-NeMo/Nemotron	사용 cookbook	Apache-2.0	2,248	2026-10-28	`99a7e45` (메타데이터만)
NVIDIA-NeMo/Automodel · RL · Gym	부모 계열의 SFT·RL·환경	Apache-2.0	1,064 · 2,095 · 1,305	2026-10-30	메타데이터만
ollama/ollama	`nemotron-3.5-bolt` 배포	MIT	185,334	2026-10-30	메타데이터만
unslothai/unsloth	미세조정·GGUF	Apache-2.0	78,795	2026-10-30	메타데이터만
scaleapi/mcp-atlas · ArtificialAnalysis/aa-agentperf-local	평가	MIT · Apache-2.0	163 · 122	2026-10	메타데이터만

공개되지 않은 것: Star Elastic 압축 코드와 라우터, QAD 학습 스크립트, 데이터 혼합 비율의 정확한 값, 평가 하네스(WebArena 내부 구현)입니다. 가중치와 런타임은 공개되어 있지만 **학습 파이프라인은 재현할 수 없습니다.**

5. 논문이 다루지 않았지만 관련 있는 것 (내 판단)

주제	관련성	근거 수준
MoE 배치 검증 비용 (예: expert offloading, speculative decoding for MoE 연구들)	초안 검증 때 토큰마다 다른 expert를 읽어 비용이 늘어나는 문제는 MoE 추론 연구에서 알려진 주제입니다. `03` §3의 toy가 같은 질문을 다룹니다.	내 해석, 서지 확인 필요
Apple MLX 저비트 양자화	맥에서는 GGUF IQ2보다 MLX 2-4비트 경로가 현실적인데, 3비트 변환도 31.65 GB라 24 GiB 맥에는 들어가지 않습니다.	HF 파일 크기로 확인

주제	관련성	근거 수준
Gemma 3n/4 E-시리즈의 PLE(층별 임베딩)	"E4B = 유효 4B"는 임베딩을 저장 장치로 옮기는 Gemma 방식이라, "4GB"가 메모리 표기인지 파일 표기인지 따로 확인해야 합니다.	내 사전 지식, 확인 필요